

The Full Stack of Innovation: Building Data & AI Products with Databricks Apps

DAIS 2025

11 June 2025



Forward-looking Statement

This presentation has been prepared for informational purposes only. The information set forth herein does not purport to be complete or contain all relevant information. Statements contained herein are made as of the date of this presentation unless stated otherwise.

This presentation and the accompanying oral commentary may contain forward-looking statements. In some cases, forward-looking statements can be identified by terms such as “may”, “will”, “should”, “expects”, “plans”, “anticipates”, “could”, “intends”, “projects”, “believes”, “estimates”, “predicts”, or “continue”, or the negative of these words or other similar terms or expressions that concern Databricks’ expectations, strategy, plans, or intentions.

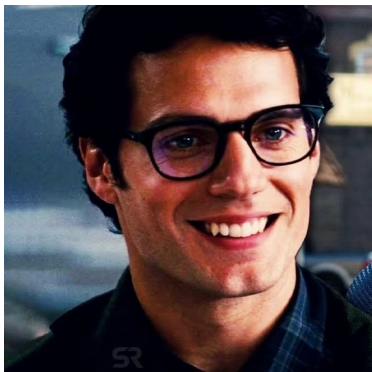
Forward-looking statements are based on information available at the time those statements are made and are inherently subject to risks and uncertainties that could cause actual results to differ materially from those expressed in or suggested by the forward-looking statements.

Forward-looking statements should not be read as a guarantee of future performance or outcomes. Except as required by law, Databricks does not undertake any obligation to publicly update or revise any forward-looking statement, whether as a result of new information, future developments or otherwise.



Learning Objectives

You are:



- Software Engineer or Application Developer without Databricks experience
- Data, ML, or AI Engineer
- Maybe not up to date with the latest Databricks offerings

You will get:



- A deep dive into Application Development with Databricks
- Understanding how to best use Databricks Apps for your organisation
- End-to-end knowledge on Databricks Apps

Agenda

- Introduction to Databricks Apps
Giran, 5 minutes
- Frameworks and Tooling
Ivan, 10 minutes
- Development Lifecycle
Giran, 10 minutes
- Integrations with Databricks APIs
Ivan, 10 minutes
- Application Testing
Giran, 10 minutes
- Deployment and CI/CD
Ivan, 10 minutes
- Best Practices and Scalability
Ivan, 10 minutes
- Monitoring and Observability
Giran, 10 minutes
- Summary and Roadmap
Giran & Ivan, 5 minutes



Introduction to Databricks Apps

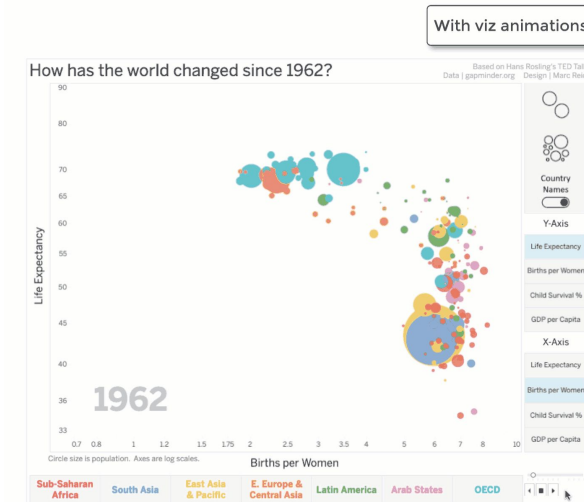
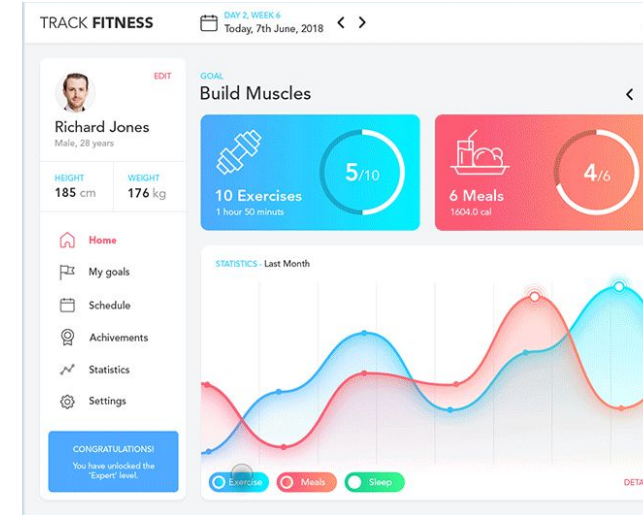
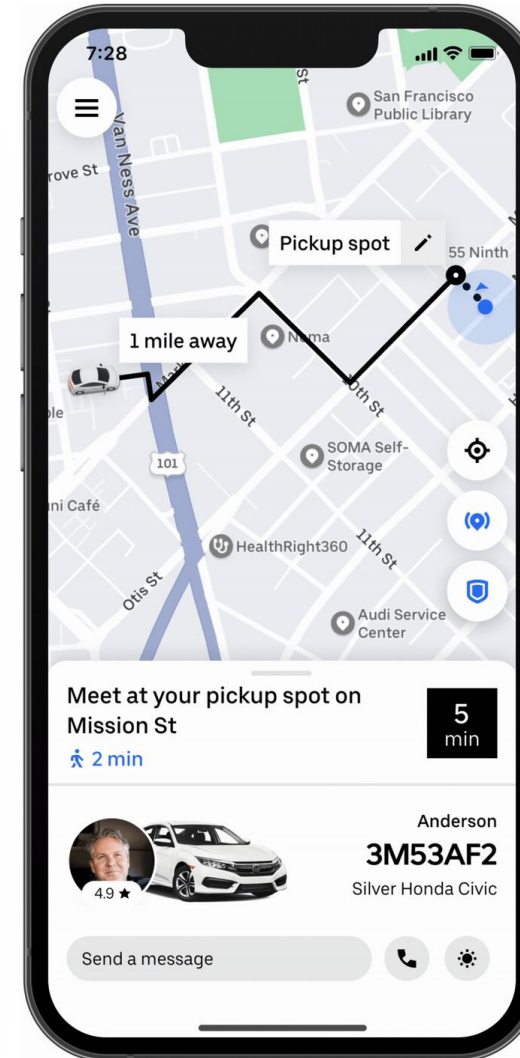


What is a Data & AI Application?

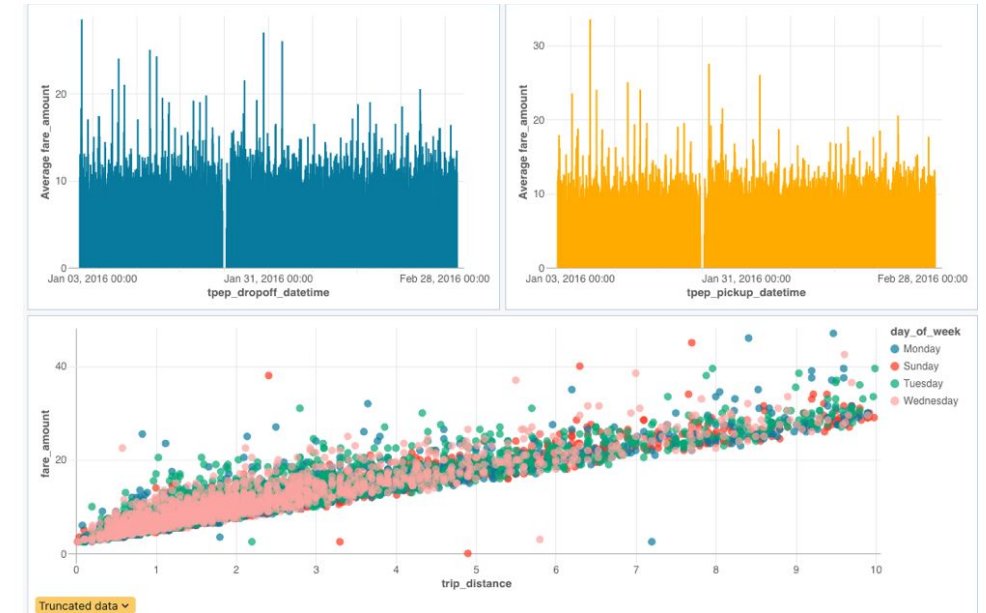
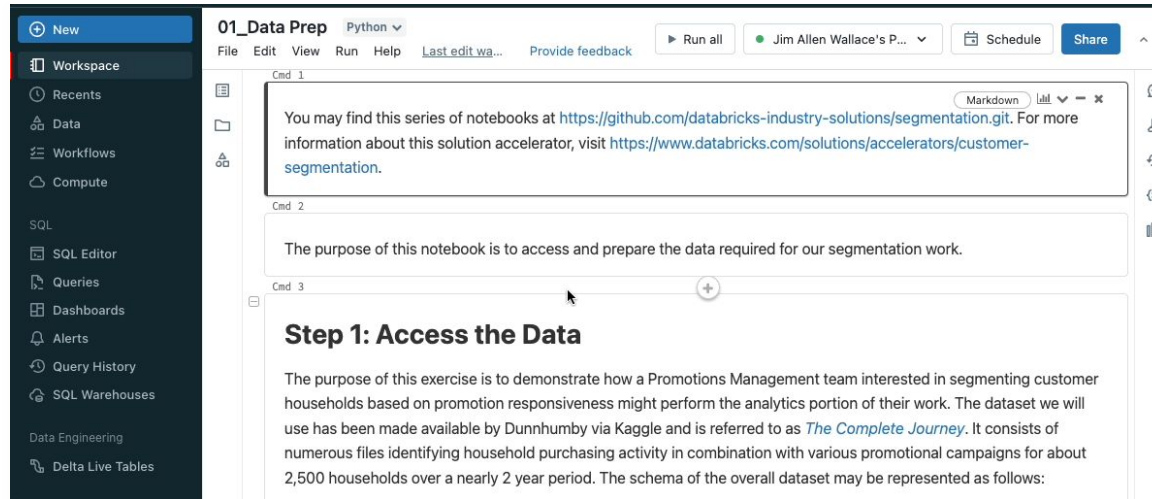
An application is an **interactive interface** that puts data and logic into hands of non-technical users.

Data & AI applications focus on:

- Revealing data for business purposes
- Connect UI to AI-driven business logic



Why apps when we have notebooks and dashboards?

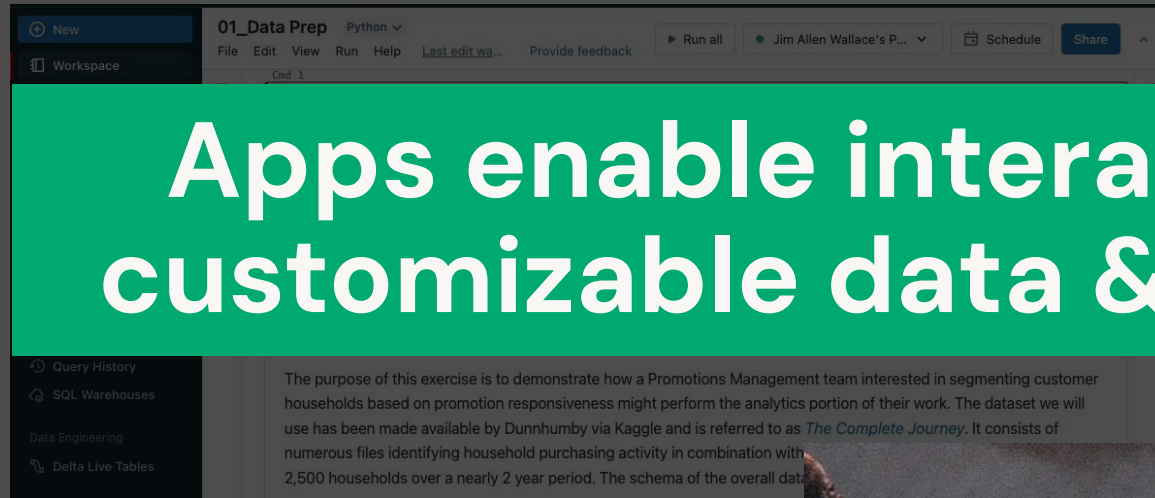


Notebooks: great for technical audience, but may seem overwhelming for non-technical users

Dashboards: great for visualization, but users require more flexibility in user interactions



Why apps when we have **notebooks** and **dashboards**?



Apps enable interactive, flexible, and customizable data & AI user interaction

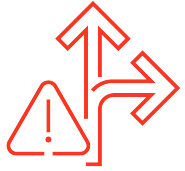
Notebooks: great for technical analysis but may seem overwhelming for non-technical users



Dashboards: great for visualization, but users lack flexibility in user interactions



Data & AI apps are hard to build and productionize



Complexity

Apps are bottlenecked around specialized DevOps resources, which slows the pace of development and makes them hard to maintain over time.



Time to delivery

Apps take longer to deploy due to increased reviews and iterations, often delaying the app's time to market. Infrastructure is extremely complex to deploy fast.



Security issues

Governing apps separately from underlying data is cumbersome and error-prone, resulting in poorly governed and insecure apps.

← All Roadmaps



Schedule Learning Time

Download

Share

Full Stack Developer

Step by step guide to becoming a modern full stack developer in 2025

Roadmap

Projects

soon

0% DONE 0 of 37 Done

What is Full Stack Development?

Start Backend Development

Backend Starts here

Redis

JWT Auth

RESTful APIs

Checkpoint — Complete App

Use the checkpoints and do not forget to practice what you learn. There are project ideas at each checkpoint that you can build to solidify your knowledge.

Full Stack

If you are already a full-stack developer you should visit the following tracks.

Frontend

Backend

DevOps

Target audience for this roadmap is absolute beginners wanting to get into full-stack development.

Find the detailed version of this roadmap along with other similar roadmaps

roadmap.sh

Key topics to learn

Project ideas and suggestions

CSS

JavaScript

Checkpoint - Static Webpages

Checkpoint - Interactivity

npm

Checkpoint - Collaborative Work

Checkpoint - External Packages

Learn CSS

GitHub

Git

Learn and Backend

Build Apps

You can pick any backend programming language. My recommendation is Node.js because you are already familiar with JavaScript and it's easier to pick.

Route53

SES

EC2

VPC

S3

Basic AWS Services

you can build to solidify your knowledge.

Checkpoint — Deployment

Monit

Checkpoint — Automation

Checkpoint — CI / CD

Checkpoint — Monitoring

Ansible

GitHub Actions

Terraform

Checkpoint — Infrastructure

**Surely there is a
better way.**

**Can we improve
this experience?**



Databricks Apps Highlights

Easy to build

Anything that can be served on the web, can be launched on Databricks Apps

Easy to operate

- Click to deploy
- Managed serverless compute in a secured environment
- Simple deployment

Easy to share

Share with non-workspace users via SCIM sync

Easy to secure

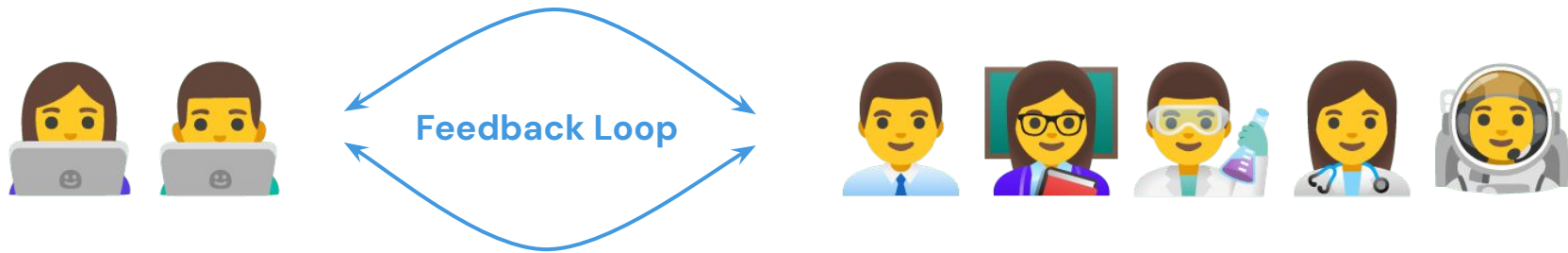
Built-in OAuth with On-Behalf-Of User and API access



Who interacts with Databricks Apps? 🔍

Built by data scientists and data teams – and software engineers!

Used by sales, marketing, finance, and other non-technical teams



Example Databricks Apps Use Cases

Interactive
data apps

ML model
interface

RAG Chatbot

Business
process
automation

Data
collection
form

Data / AI
review &
approval

What-if
analysis

GenAI agents

Complex
visualizations

No-code
application
builder

UI for APIs

and much more...

Apps are cool!

**But what about
tooling?**



Frameworks and Tooling



Tooling Overview

What we will cover

With Databricks Apps you can do anything, but common choices are:

- Pythonic Frameworks (Dash, Streamlit, ...)
- JS frontend frameworks (React, Vue, Svelte) + Python backend
- [**new**] NodeJS backend!



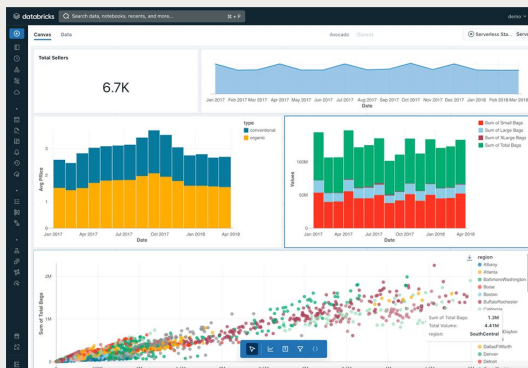
Choosing the right tool for your use case



AI/BI

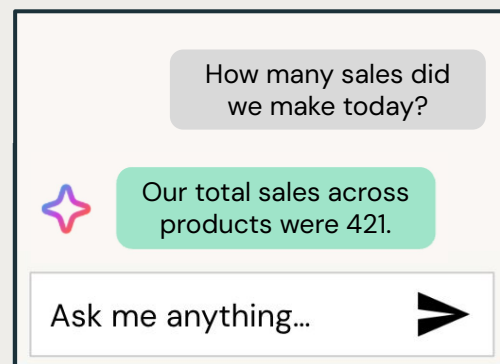
Dashboards

No-code data visualization



Genie

Chat with your data



Databricks Apps

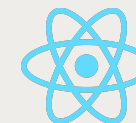
App frameworks

Build with predefined app components



Custom applications

Run any Python-based application



More control and customization <> Higher implementation effort and complexity

Templates

Also available as GitHub repo!

Compute > Apps >

Create new app

Choose how to start

Template
Quickly create an app using the template library

Custom
Use your existing source code and packages to create an app

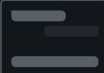
Dash

Flask

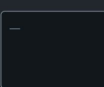
Gradio

Shiny

Streamlit

**Chatbot**
Create a chat UI using a large language model and Model Serving.
Dependencies: Serving endpoint

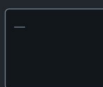
**Data app**
Create an app that reads from a SQL warehouse and visualizes the data.
Dependencies: SQL warehouse

**Hello world**
Create a simple app.

Dash **Gradio** Shiny Streamlit

**Chatbot**
Create a chat UI using a large language model and Model Serving.
Dependencies: Serving endpoint

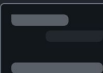
**Data app**
Create an app that reads from a SQL warehouse and visualizes the data.
Dependencies: SQL warehouse

**Hello world**
Create a simple app.

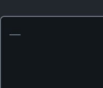
Dash **Flask** Gradio Shiny Streamlit

**Hello world**
Create a simple app.

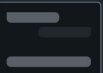
Dash Flask Gradio **Shiny** Streamlit

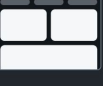
**Chatbot**
Create a chat UI using a large language model and Model Serving.
Dependencies: Serving endpoint

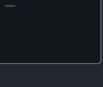
**Data app**
Create an app that reads from a SQL warehouse and visualizes the data.
Dependencies: SQL warehouse

**Hello world**
Create a simple app.

Dash Flask Gradio Shiny **Streamlit**

**Chatbot**
Create a chat UI using a large language model and Model Serving.
Dependencies: Serving endpoint

**Data app**
Create an app that reads from a SQL warehouse and visualizes the data.
Dependencies: SQL warehouse

**Hello world**
Create a simple app.



Pythonic or JS-driven UI?

What should I pick?

Python

- Frameworks:
 - Dash, Streamlit, Reflex, Gradio
- All have pythonic APIs
- Tight integration with server-side logic
- Support custom components
- For full customization might still require writing JS/JSX code

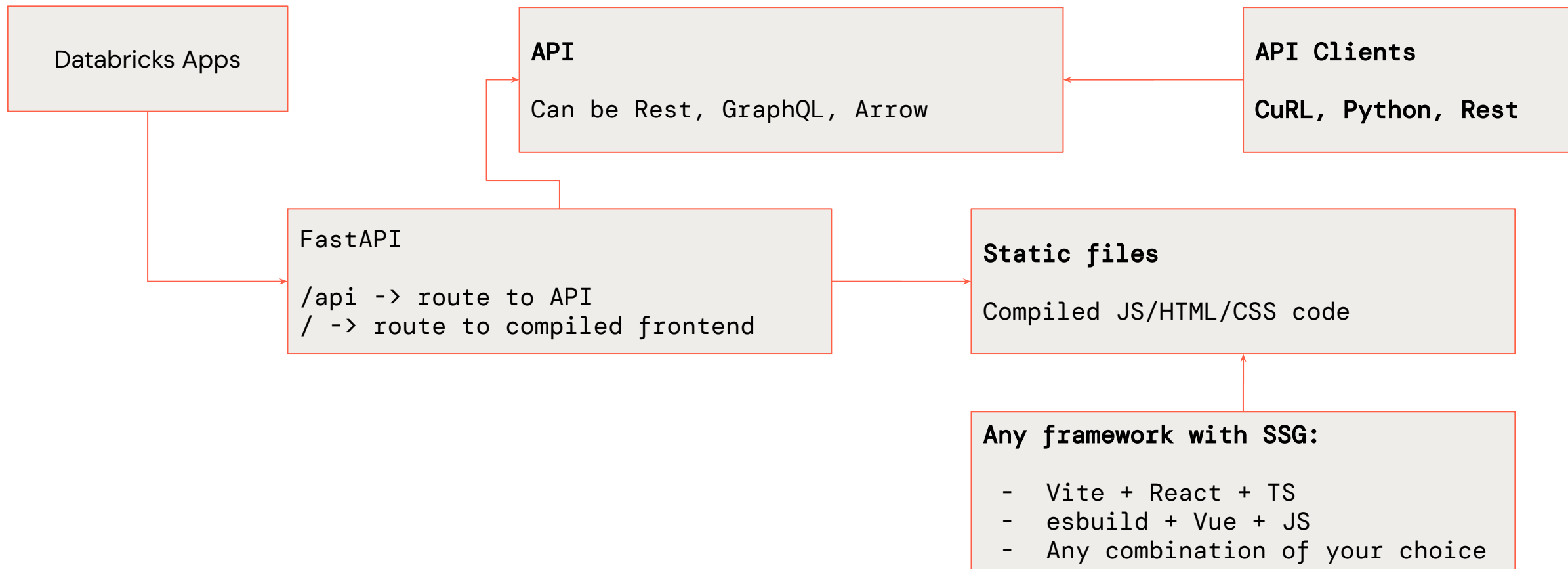
JS

- Frameworks:
 - React, Vue, Angular, Svelte and many others
- Ready-to-use layout syntax (JSX/Svelte)
- Would require certain knowledge of JS/TS and JSX
- Provide full customization capabilities
- Flexible styling with CSS
- Better client-side performance



JS frontend + Python backend

Flexible and powerful combination



Pure API Apps

Apps with backend-only routes

```
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
```

```
app = FastAPI()
```

```
class Item(BaseModel): # Item model
    name: str
    price: float
```

```
@app.post("/api/items/{item_id}") # Create
@app.get("/api/items/{item_id}") # Read
@app.put("/api/items/{item_id}") # Update
@app.delete("/api/items/{item_id}") # Delete
```

- Build a business layer logic on top of Databricks
- Simple OAuth integration
- Works natively without extra configuration effort



**Slides are good,
can we see it in
action?**



Demo

FastAPI Backend

React Frontend



**How can I develop
an app myself?**



Development Lifecycle



Section Overview

What we will cover

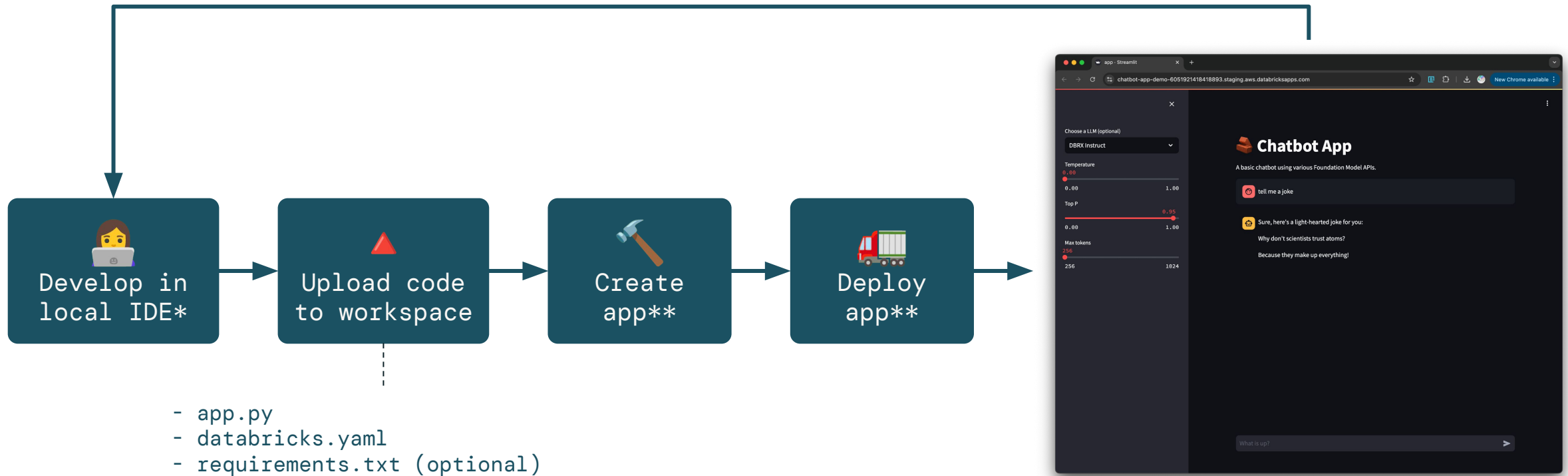
- Pythonic development with Dash
- Application environment and access to resources
- Debugging and troubleshooting



Developing with Dash



[Minimal] Path to Deployment

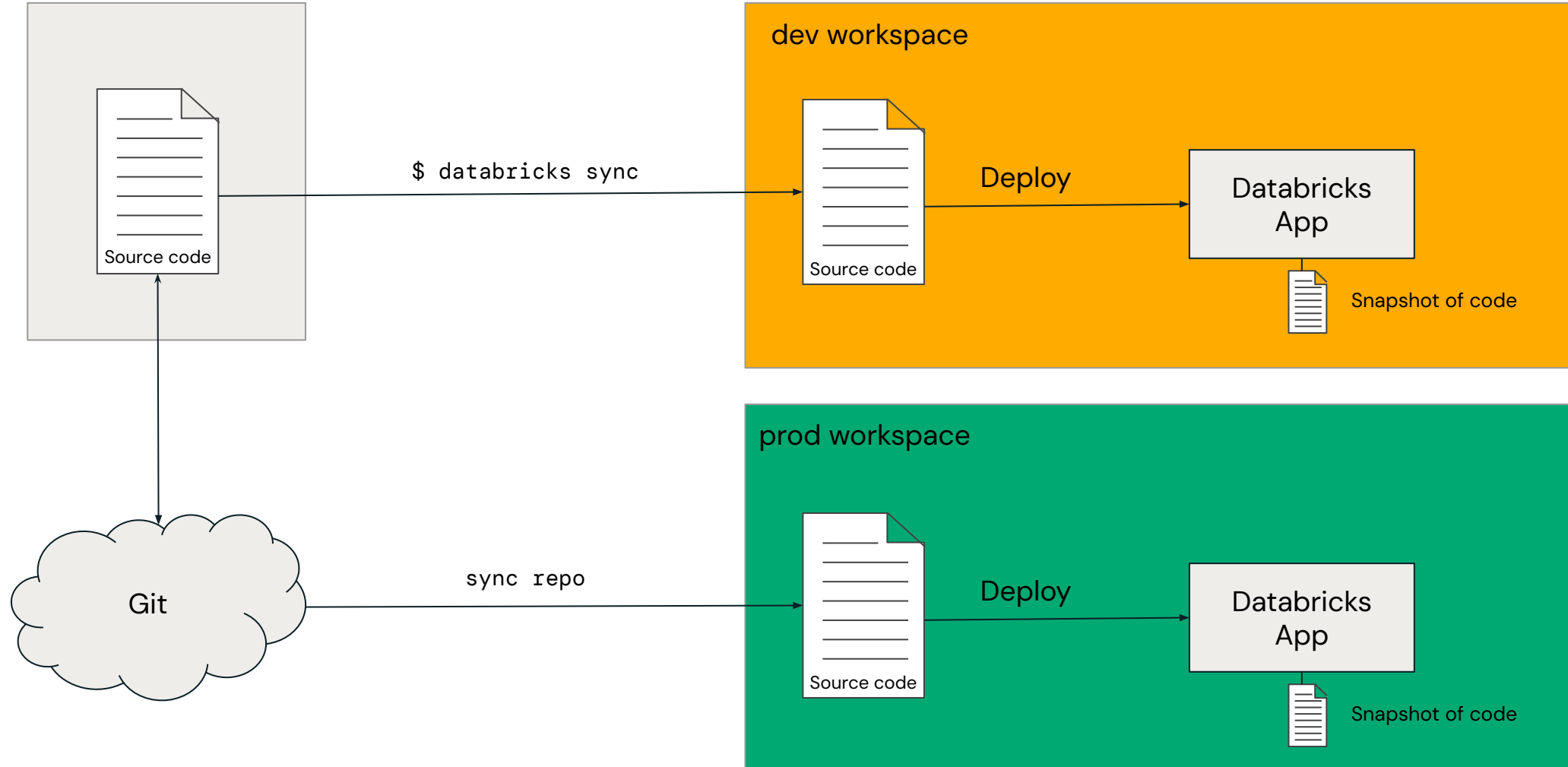


NB: This is minimal example, more profound CI/CD setup will be show in later slides

* workspace editor also OK but less preferred
** APIs available for automation & CI/CD purposes



Typical Code Flow Between Environments



Example: Data App from Dash Template

Compute > Apps >

Create new app

Create custom app from scratch

Create a custom app
Bring your code and dependencies

Install from a template

Dash

Flask

Gradient Boosting

Chatbot

Create a chat UI using a large language model and Model Serving.

Dependencies: Serving endpoint

Data app

Create an app that reads from a SQL warehouse and visualizes the data using the app user's permissions.

1 Configure resources

2 Review authorizations

3 Add metadata

This app requires the following resources to function properly. Select the resources to grant access to the app.

SQL warehouse

The SQL warehouse the app uses to query data.

Serverless Starter Warehouse

Back

Next

Compute > Apps >

Install App

Configure resources

Review authorizations

Add metadata

Once installed, this app will gain the following authorizations.

App authorization

The app's service principal will access the following resources on behalf of the app.

Resource

Permission

Serverless Starter Warehouse

Can use

User authorization

The following user authorized API scopes are required in order to access resources on behalf of the user. Users must consent to these scopes before using the app for the first time.

Scope

Description

sql

Allow the app to execute SQL and manage SQL related resources in Databricks.

Back

Next

Compute > Apps >

Install App

Configure resources

Review authorizations

Add metadata

App name

Name can't be changed once app is created

gm-ui-dash-data-app

Description

A starter dashboard application which queries data in Unity Catalog with a SQL Warehouse using the app user's permissions.

Tags

Back

Install

taxi

Taxi Fare Distribution

Fare

Distance

Predict Fare

From (zipcode)

10003

To (zipcode)

11238

Predict

Predicted Fare: \$99.00

tpep_pickup_datetime	tpep_dropoff_datetime	trip_distance	fare_amount	pickup_zip
2016-02-13T21:47:53+...	2016-02-13T21:57:15+...	1.4	8	10103
2016-02-13T18:29:09+...	2016-02-13T18:37:23+...	1.31	7.5	10023

databricks

Permission Requested

gm-ui-dash-data-app

giran.moodley@databricks.com

This app is requesting permission to act on your behalf

Databricks SQL

By clicking Authorize, you allow this app to take actions on your behalf. Proceed only if you trust this application and its author.

Cancel

Authorize

Compute > Apps >

gm-ui-dash-data-app

Send feedback

Edit

Permissions

Deploy

Overview

Authorization

Deployments

Logs

Environment

Running

https://gm-ui-dash-data-app-2866853758957189.aws.databricksapps.com

Deployment

Source: /Workspace/Users/giran.moodley@databricks.com/databricks_apps/gm-ui-dash-data-app_2025_06_10

Last deploy completed 58 seconds ago

Inspect deployed code

Jun 10, 2025, 10:12:09 AM PDT

Stopped app successfully

Jun 10, 2025, 10:12:10 AM PDT

Source code downloaded successfully

Jun 10, 2025, 10:12:10 AM PDT

App spec validated successfully

Jun 10, 2025, 10:12:11 AM PDT

Requirements installed successfully

Jun 10, 2025, 10:12:13 AM PDT

App started successfully



Demo

Creating a Databricks app from a
template





Where is my application running?



App environment details

Resources

- Container in the Databricks Serverless Plane
- 2 vCPU & 6GB RAM

NB: Complex/memory intensive workloads best serviced by Clusters/Warehouses

Container

- Ubuntu 22.04
- Python 3.11
- Isolated virtual env
- Code sourced from Workspace
- Install any extra packages via requirements.txt



How to debug issues?

Code failures

- Databricks Apps cannot (yet) fix logical issues in your code
- Use Copilot or any other coding assistant
- Apps should be tested locally before deploying
- Use standard local IDE tooling for debugging
- Add verbose logs to catch upon specific behavior

Env configuration

- Check env variables validity at startup
- [JS/TS apps] Make sure CORS is configured properly
- **No need for PATs.**
- Use `WorkspaceClient` and it will work like a charm

Startup command

- Run this in a local docker container
- Make sure all relevant files are uploaded
- Make sure command runs in the correct folder
- Avoid relative paths



**My app needs
data and AI
services!**

**How can I use
them?**

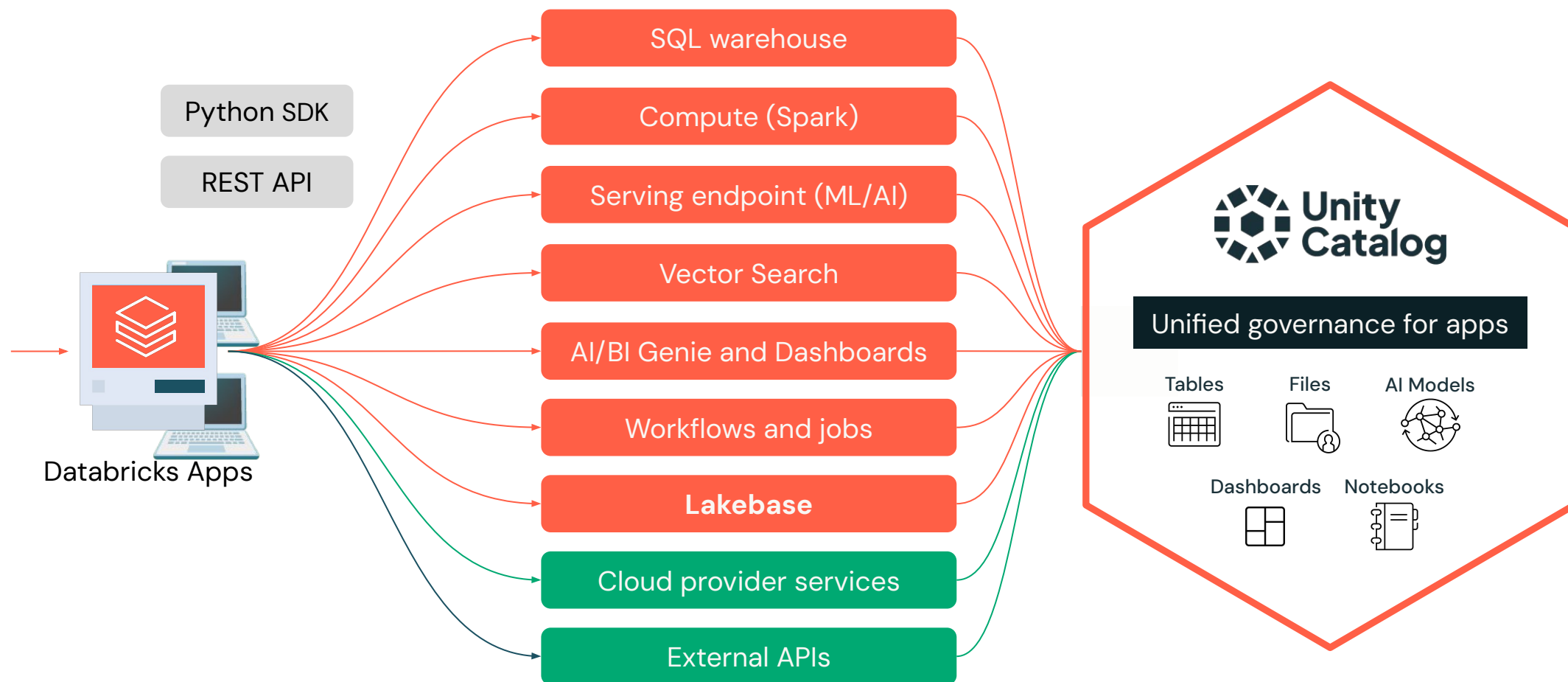


Integration with Databricks APIs



Apps integrate with your data and AI stack

Connect seamlessly with internal and external services



App principal and user identity

Service Principal

- Created by default for every app
- Can be used to perform actions outside of user access context
- Can be granted with access to relevant resources

You can combine both SP and user clients in the same app for different actions

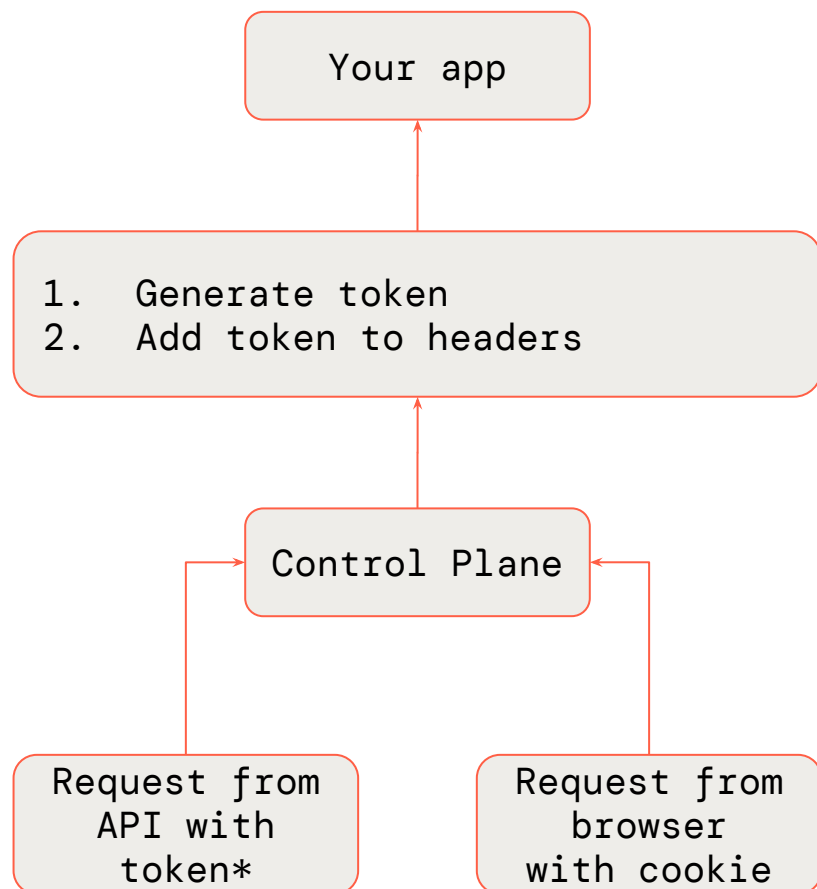
User identity

- Passed via headers into the app
- Used for OBO (on-behalf-of) inside the app

Use OBO for user-based data/AI access



OBO – how it works?



Your application:

1. Pick the token from header
2. Use it in the workspace client

```
@app.get("/me")
async def me(
    token: Annotated[
        str,
        Header(alias="X-Forwarded-Access-Token")
    ] -> JSONResponse:
    as_user = WorkspaceClient(token=token)
    user = as_user.current_user.me()
    return user.as_dict()
```

* Token must have same scopes as the app



Using Databricks SDK tooling with OBO

SQL connector

```
from databricks import sql

def get_table(token: str):
    # Connect to Databricks
    with sql.connect(
        access_token=token
    ) as conn:
        cursor = conn.cursor()
        cursor.execute("SELECT name, age FROM users")
        # fetch data
        return cursor.fetchall_arrow()
```

API

```
from databricks.sdk import WorkspaceClient

def get_prediction(token: str, inputs):
    w = WorkspaceClient(token)

    # Call a serving endpoint
    response = w.serving_endpoints.query(
        name="endpoint-name"
        inputs={...}
    )
```



**Before revealing
app to users, I
need to test it!**

How can I do this?



Application Testing



Section Overview

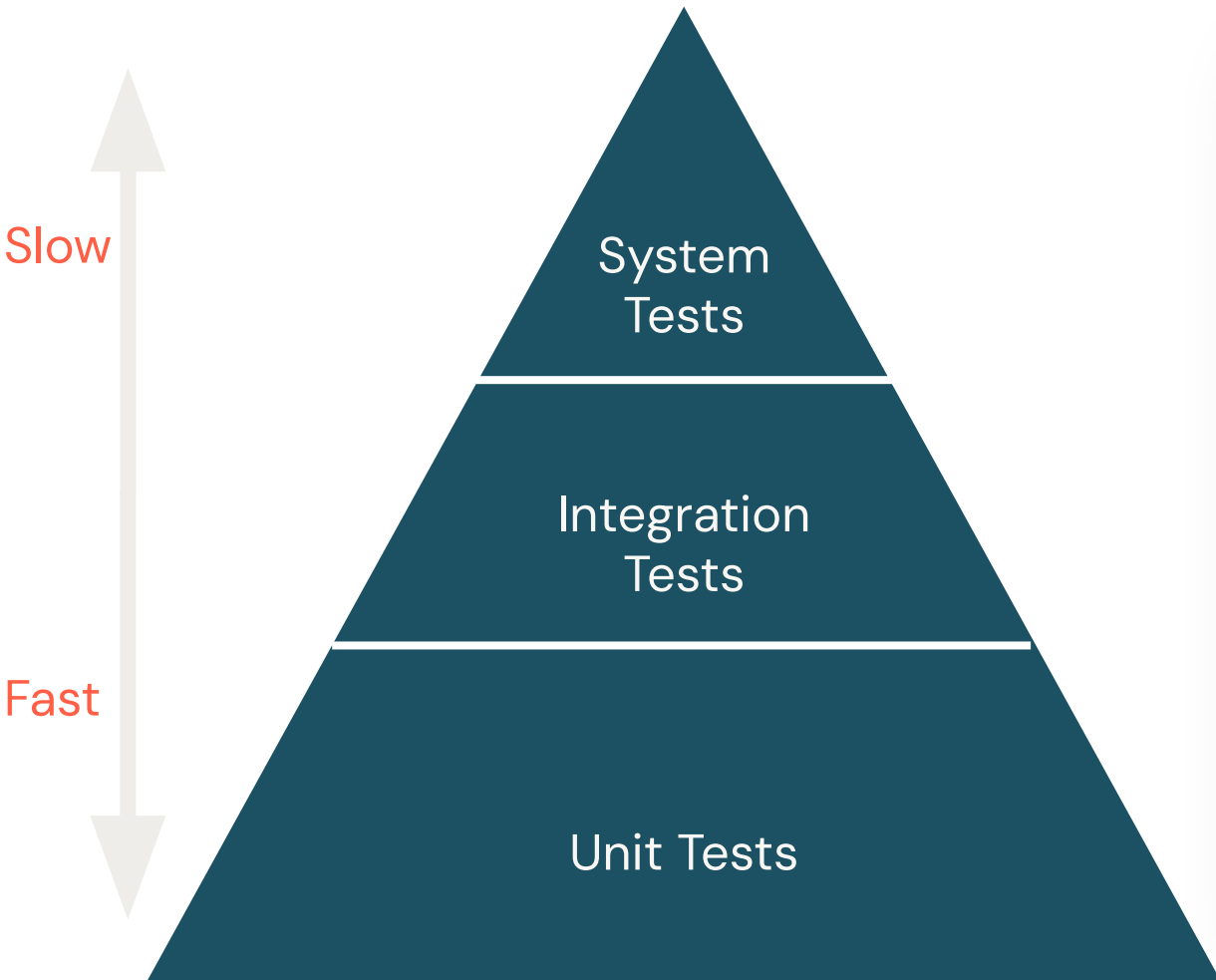
What we will cover

- Testing pyramid
- Different types of testing and relevant tooling
- Demo



Testing Pyramid

Ensuring our application works as intended



Dev lifecycle:

- **System tests** - verify e2e flow (including upstream and downstream dependencies)
- **Integration tests** - verify that application runs in the prod-like environment
- **Unit tests** - intended for quick devloop interactions

Additionally:

- Load testing
- Manual / user testing

Testing Frameworks & Utilities

Common libraries used

Python Ecosystem

- Pytest (as a main framework)
- Faker (generating mock data)
- Playwright (UI testing)

JavaScript Ecosystem

- E2E:
 - Playwright
 - Cypress
- Integration:
 - Jasmine
 - Enzyme
- Unit:
 - Jest

Load Testing

- Locust
- Gatling
- JMeter



Demo

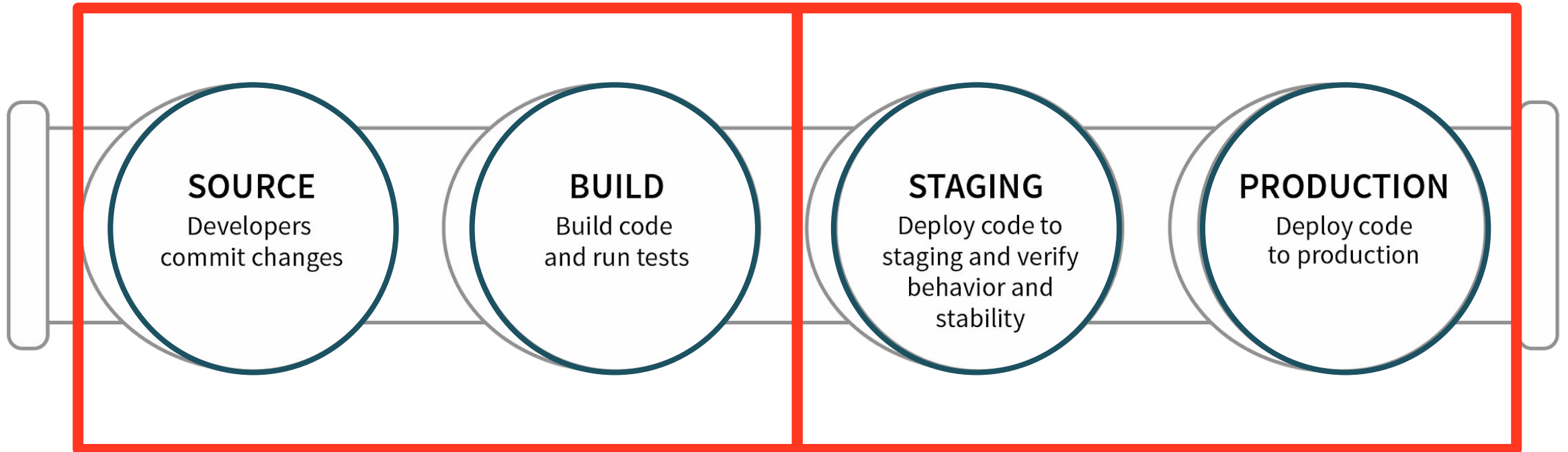
Example Databricks App with
Pytest, Playwright, and Locust



Deployment and CI/CD



CI/CD basics



Continuous Integration (CI)

Continuous Delivery and/or
Continuous Deployment (CD)



Databricks Asset Bundles for Apps

- Simple and declarative deployment pattern
- CLI interface for any CI/CD provider



This app is ready for production, time to deploy!

```
$ databricks bundle deploy -t prod
```

```
bundle:
  name: "databricks-app-bundle"

resources:
  apps:
    awesome-app:
      name: "awesome-app"
      description: "My awesome app"
      resources:
        - name: "sql-warehouse-resource"
          description: "SQL Warehouse for App"
          sql_warehouse:
            id: "warehouse_id"
            permission: "CAN_USE"

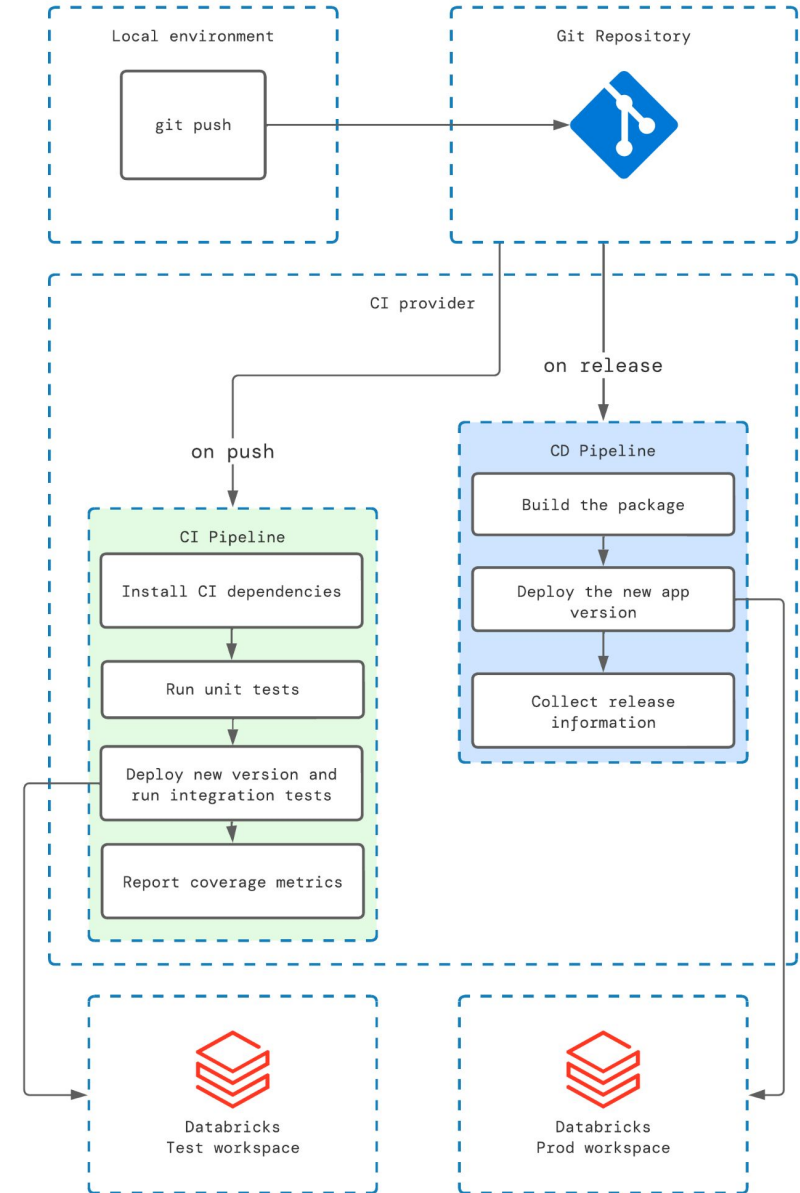
      permissions:
        - group_name: "analysts"
          permission_level: "CAN_USE"
        - group_name: "admins"
          permission_level: "CAN_MANAGE"

  targets:
    dev:
      default: true
      workspace:
        host: "https://<dev-workspace-url>"
    prod:
      workspace:
        host: "https://<production-workspace-url>"
```



General CI/CD pattern

- Keep your application code in git
- Use DABs to define application resources and overall setup
- Use Databricks CLI in CI/CD pipelines
- [CI]
 - Work in feature branches
 - Merge into main when ready
- [CD]:
 - Trigger a release pipeline with release tag



CI/CD tooling and setup

Python packaging

- Package managers:
 - **uv** - via custom build hook*
 - **requirements.txt** - supported
 - **poetry/hatch** - via custom build hook
- Use self-hosted PyPi registries if applicable
- Separate code into application logic and common library

* native uv support is on the way

CI/CD setup

- Databricks CLI
- Databricks DABs with variables to support environment properties
- Use self-hosted runners for secure network access to Databricks

* simplified Git-based hook is on the way



Demo

CI/CD for a Databricks App
with Github Actions



**My app is
deployed and now
I have 100+ users!**

**What about
scalability?**



Best Practices and Scalability



Development and deployment

Development

- Run the app locally with the correct Python version 
- Connect to Databricks using OAuth (**no PAT tokens**) 
- Use testing frameworks like `pytest` & `playwright` to validate behavior 

Deployment

- Use Databricks Asset Bundles (DABs) to deploy apps & workflows 
- CI pipeline for unit tests 
- CD pipeline to deploy the app 
- Separate environments (e.g., dev & prod) 

 **Use OBO tokens to act on behalf of a user's identity** 

Runtime and performance

- Use **async** operations in DBSQL connector
 - For heavy UI customization, use JS/TS frontend + Python backend 
 - Prefer “status pull” over long-running tasks / connections
- Pin package versions (using uv or any preferred manager) 
 - For Pythonic frontends, choose UI frameworks that support multi-worker setups 
 - Use a process manager (e.g. uvicorn) to improve stability and performance

**I've prepared my
app and it scales!**

**How can I monitor
the app behavior?**



Monitoring and Observability



Section Overview

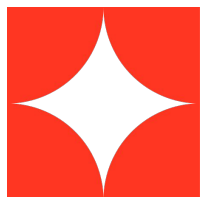
What we will cover

- Monitoring vs Observability
- Monitoring Databricks Apps
 - App logs
 - Audit logs
- Observability with OpenTelemetry
 - Metrics & Tracing
- Dashboarding & Alerting
 - Collect data with opentelemetry – push to Delta
 - AI/BI Dashboards & Genie to understand
 - Alerting & Notifications to respond in a timely manner



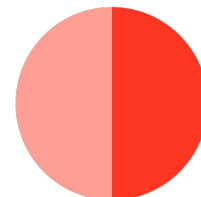
Monitoring vs Observability

As Dr Dre once said – “What’s the difference?”



Monitoring ~ “What & When?”

- **Answers:** "Something is wrong"
- Tracks **known metrics** and predefined thresholds
- **Reactive** approach – alerts when problems occur
- Focuses on **individual components**
- **Symptom detection** – identifies that issues exist



Observability ~ “Why & How?”

- **Answers:** "Why it's happening & how to fix it"
- Investigates **unknown issues** and root causes
- **Proactive** approach – prevents problems before they occur
- Focuses on **distributed systems** as a whole
- **Root cause analysis** – diagnoses underlying causes

Monitoring tells you **that** something is wrong, while observability tells you **what** is wrong and **why** it happened

Monitoring Databricks Apps

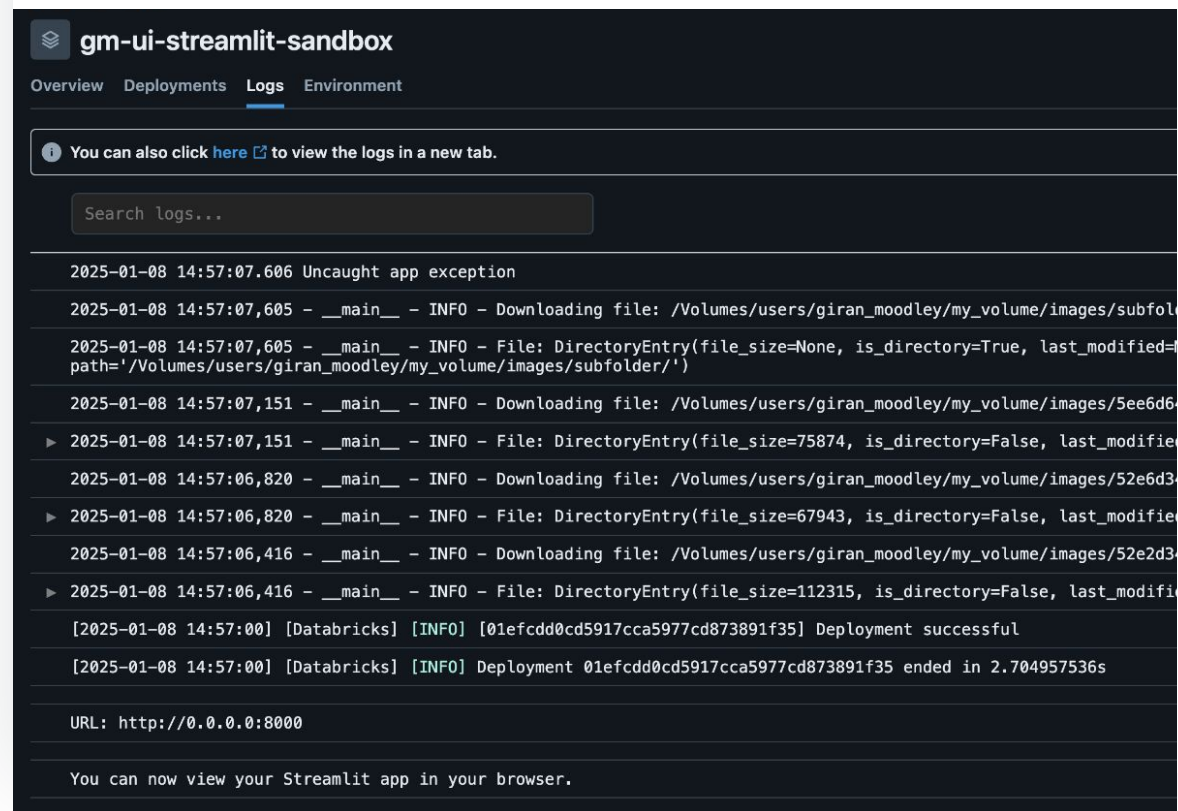


App Logs

D diagnostic Information

- Real-time logs of app from stdout and stderr; showing pip, deployment, and app details, ***ordered by most recent***.
- Recommended to configure logging library as needed (example on RHS)
- Available in standalone page via <app-url>/logz
- Erased when compute is stopped
- Secrets are redacted

```
# Example logging configuration
import logging
logging.basicConfig(level=logging.INFO, format="%(asctime)s | %(name)s |
%(levelname)s | %(message)s")
logger = logging.getLogger(__name__)
logger.info("Logging some event")
```



The screenshot shows the 'gm-ui-streamlit-sandbox' interface with the 'Logs' tab selected. It displays a list of log entries with timestamps, log levels, and messages. The logs include information about file downloads, directory entries, and deployment success. A search bar is present at the top of the log list.

gm-ui-streamlit-sandbox

Overview Deployments **Logs** Environment

You can also click [here](#) to view the logs in a new tab.

Search logs...

2025-01-08 14:57:07.606 Uncaught app exception

2025-01-08 14:57:07,605 - __main__ - INFO - Downloading file: /Volumes/users/giran_moodley/my_volume/images/subfol

2025-01-08 14:57:07,605 - __main__ - INFO - File: DirectoryEntry(file_size=None, is_directory=True, last_modified=

path='/Volumes/users/giran_moodley/my_volume/images/subfolder/')

2025-01-08 14:57:07,151 - __main__ - INFO - Downloading file: /Volumes/users/giran_moodley/my_volume/images/5ee6d6

▶ 2025-01-08 14:57:07,151 - __main__ - INFO - File: DirectoryEntry(file_size=75874, is_directory=False, last_modifie

2025-01-08 14:57:06,820 - __main__ - INFO - Downloading file: /Volumes/users/giran_moodley/my_volume/images/52e6d3

▶ 2025-01-08 14:57:06,820 - __main__ - INFO - File: DirectoryEntry(file_size=67943, is_directory=False, last_modifie

2025-01-08 14:57:06,416 - __main__ - INFO - Downloading file: /Volumes/users/giran_moodley/my_volume/images/52e2d3

▶ 2025-01-08 14:57:06,416 - __main__ - INFO - File: DirectoryEntry(file_size=112315, is_directory=False, last_modifi

[2025-01-08 14:57:00] [Databricks] [INFO] [01efcdd0cd5917cca5977cd873891f35] Deployment successful

[2025-01-08 14:57:00] [Databricks] [INFO] Deployment 01efcdd0cd5917cca5977cd873891f35 ended in 2.704957536s

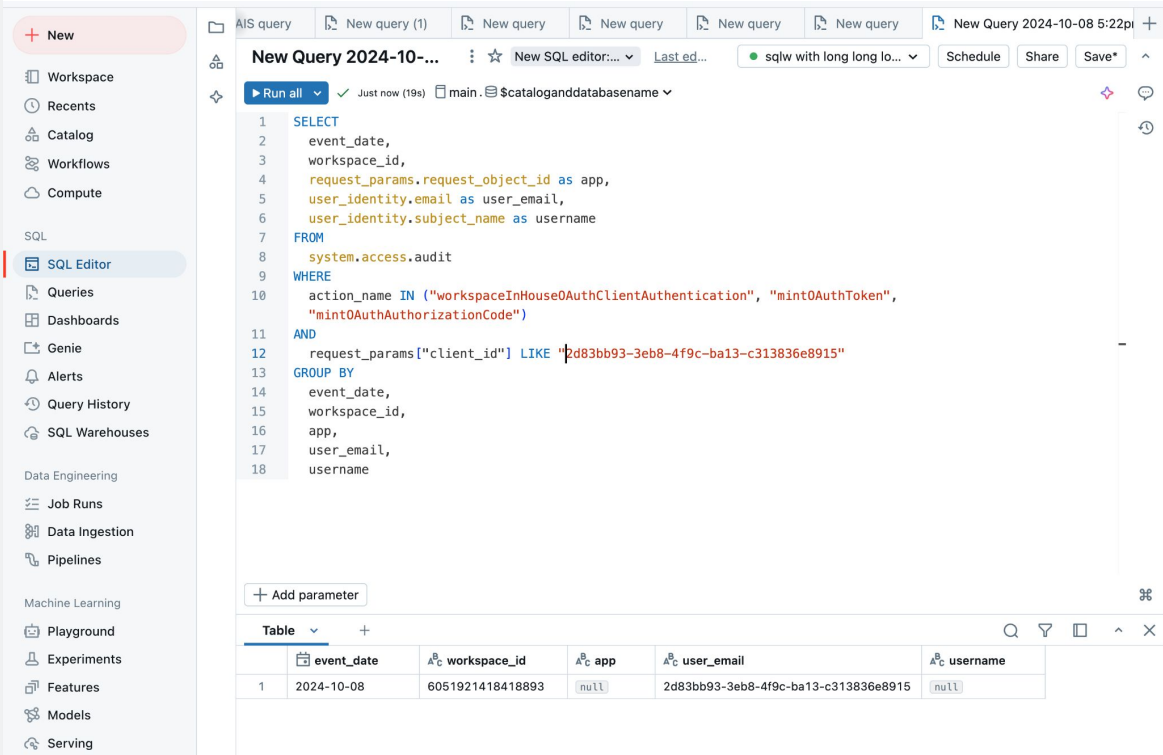
URL: http://0.0.0.0:8000

You can now view your Streamlit app in your browser.

Audit Logs

FROM `system.access.audit`

- Logins to an app [[docs](#)]
- Permission or sharing changes [[docs](#)]
- Complements Databricks AI Security Framework [[DASF](#)]



The screenshot shows the Databricks SQL interface. On the left is a sidebar with navigation options: New, Workspace, Recents, Catalog, Workflows, Compute, SQL (selected), SQL Editor, Queries, Dashboards, Genie, Alerts, Query History, SQL Warehouses, Data Engineering, Job Runs, Data Ingestion, Pipelines, Machine Learning, Playground, Experiments, Features, Models, and Serving. The main area displays a SQL query editor for a new query titled 'New Query 2024-10-...'. The query is as follows:

```
1 SELECT
2   event_date,
3   workspace_id,
4   request_params.request_object_id as app,
5   user_identity.email as user_email,
6   user_identity.subject_name as username
7 FROM
8   system.access.audit
9 WHERE
10  action_name IN ("workspaceInHouseOAuthClientAuthentication", "mintOAuthToken",
11                 "mintOAuthAuthorizationCode")
12 AND
13  request_params["client_id"] LIKE "2d83bb93-3eb8-4f9c-ba13-c313836e8915"
14 GROUP BY
15   event_date,
16   workspace_id,
17   app,
18   user_email,
19   username
```

Below the query editor, there is a table view showing the results of the query. The table has columns: event_date, workspace_id, app, user_email, and username. The first row of data is:

	event_date	workspace_id	app	user_email	username
1	2024-10-08	6051921418418893	null	2d83bb93-3eb8-4f9c-ba13-c313836e8915	null

Observability with OpenTelemetry



OpenTelemetry (OTel)

An open-source observability framework for collecting telemetry data

What is the OpenTelemetry Protocol?

- Provides standardized tools and APIs for instrumenting applications
- Vendor-neutral solution for generating, collecting, and exporting telemetry data
- Supports multiple programming languages and platforms
- Combines traces, metrics, and logs in a unified data collection approach

Why OTel?

OTel satisfies the need for observability while following two key principles:

- You own the data that you generate. There's no vendor lock-in.
- You only have to learn a single set of APIs and conventions.

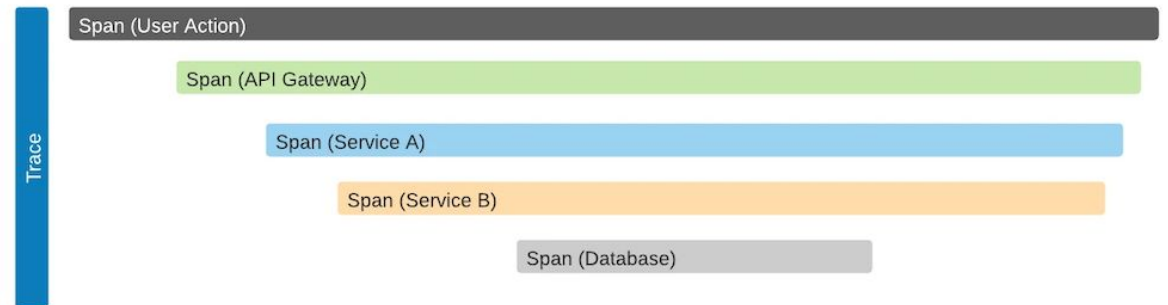
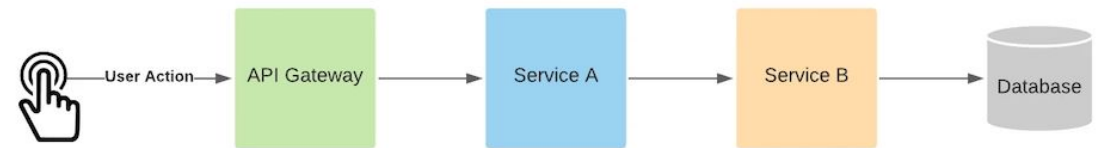
Both principles combined grant teams and organizations the flexibility they need in today's modern computing world.



Tracing

Pinpoint application behaviour throughout the entire lifecycle

- Tracing is a method of tracking a request's journey through a distributed system, providing visibility into the flow and performance of operations across multiple services or components.
- It helps developers understand the path of a request, identify bottlenecks, and diagnose issues in complex, microservices-based architectures.
- Key Components
 - Spans
 - Context Propagation
 - Distributed Tracing



```
# Acquire a tracer
tracer = trace.get_tracer("diceroller.tracer")

app = Flask(__name__)

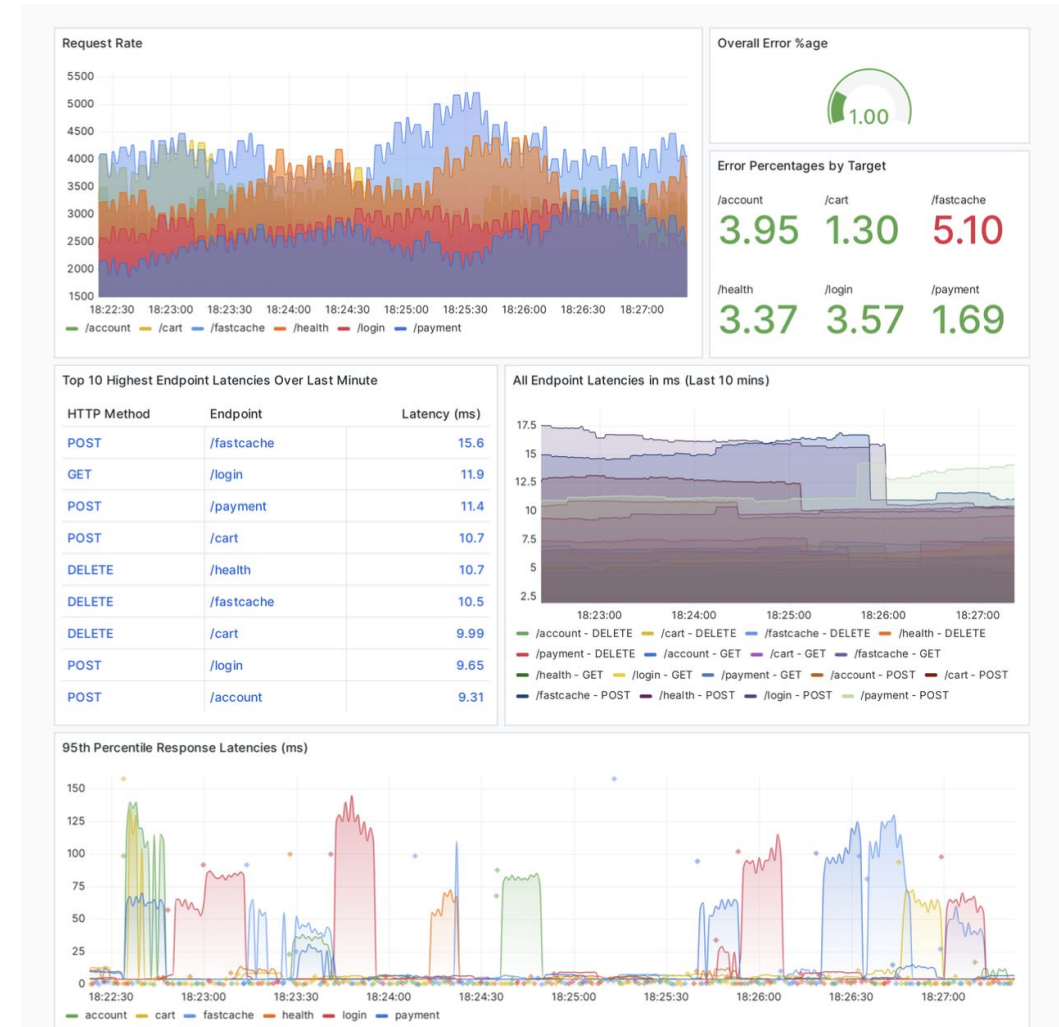
@app.route("/rolldice")
def roll_dice():
    return str(roll())

def roll():
    # This creates a new span that's the child of the current one
    with tracer.start_as_current_span("roll") as rollspan:
        res = randint(1, 6)
        rollspan.set_attribute("roll.value", res)
    return res
```

Metrics

Quantify application behaviour, analyze, and remediate

- Metrics are numerical measurements of system behavior and performance collected over time, providing quantitative insights into the health and status of applications and infrastructure.
- They provide a high-level overview of system performance, helping to identify trends, anomalies, and potential issues before they become critical problems.
- Types of Metrics:
 - Counters
 - Gauges
 - Histograms



Dashboarding & Alerting



Export to ~~Excel~~ S3

Leverage the power of cloud object storage

- The OTel collector offers a number of ways to export data to target systems.
- *Example:* AWS S3 can be used to receive data in the OTel specification as JSON.
- Databricks frameworks enable ingestion of OTel data into Delta UniForm for further analytics.

Following example configuration defines to store output in 'eu-central' region and bucket named 'databucket'.

```
exporters:
  awss3:
    s3uploader:
      region: 'eu-central-1'
      s3_bucket: 'databucket'
      s3_prefix: 'metric'
      s3_partition: 'minute'

  # Optional (disabled by default)
  sending_queue:
    enabled: true
    num_consumers: 10
    queue_size: 100
```

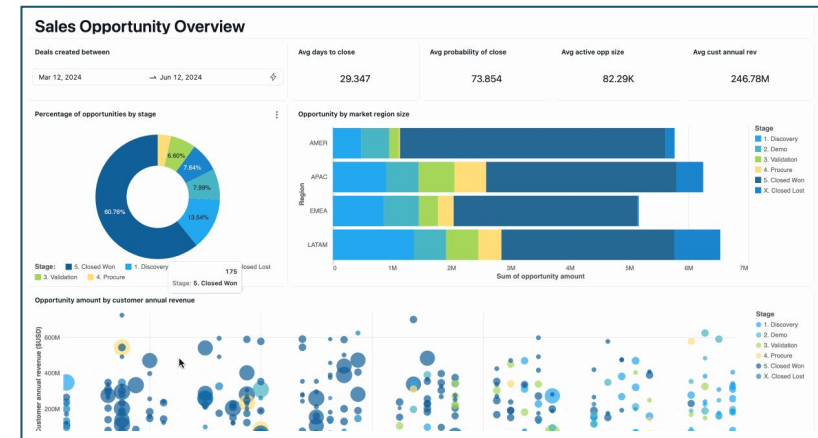
Logs and traces will be stored inside 'databucket' in the following path format.

```
metric/year=XXXX/month=XX/day=XX/hour=XX/minute=XX
```

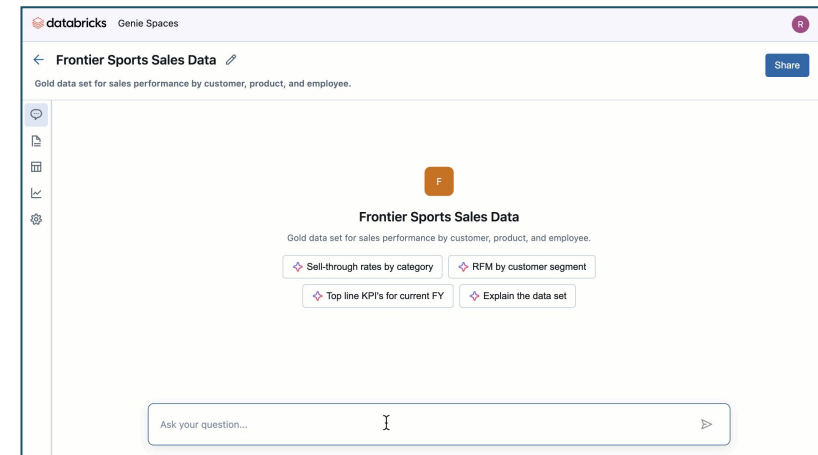
Databricks AI/BI

Intelligent analytics
for real-world data

Generally Available
Dashboards



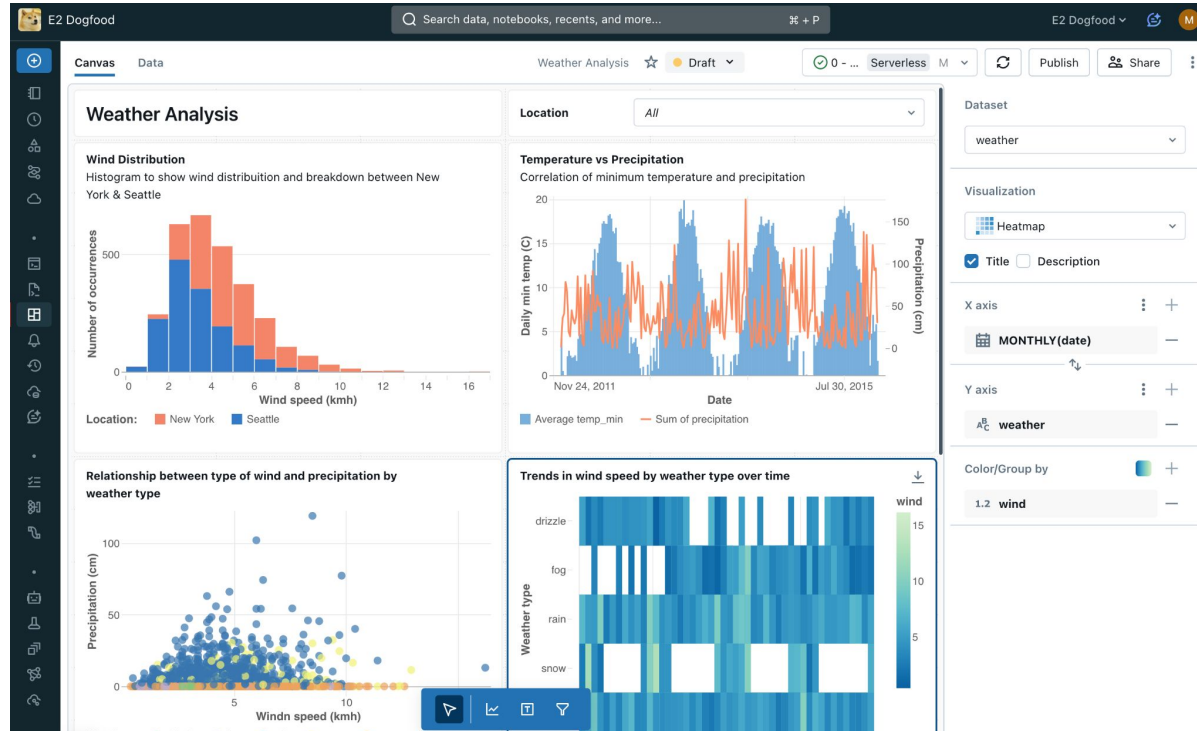
Public Preview
Genie



Governed and Secured with Unity Catalog



AI/BI Dashboards



Simple and Beautiful

Simplified content model, improved visualizations and AI-augmented UX

Optimized for Distribution

Publish and Share to business users & groups
Embedding

Platform Integration

AI functions

Unity Catalog powered dataset search and lineage

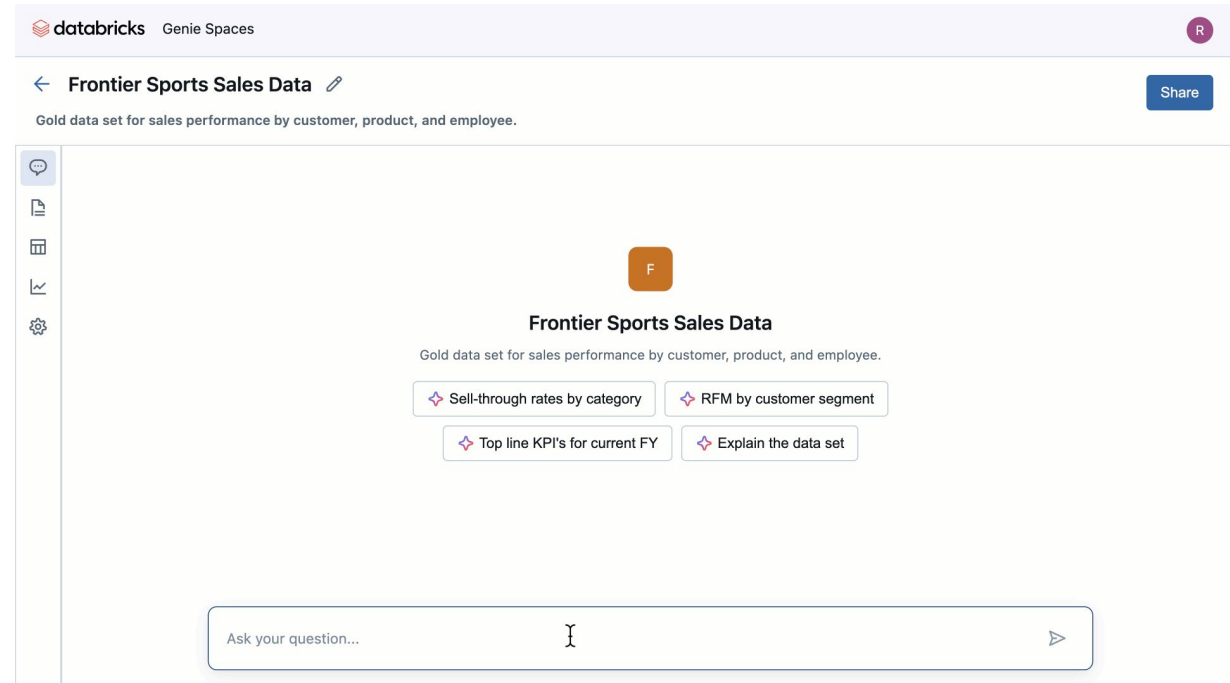


AI/BI Genie

Ask Questions in Natural Language

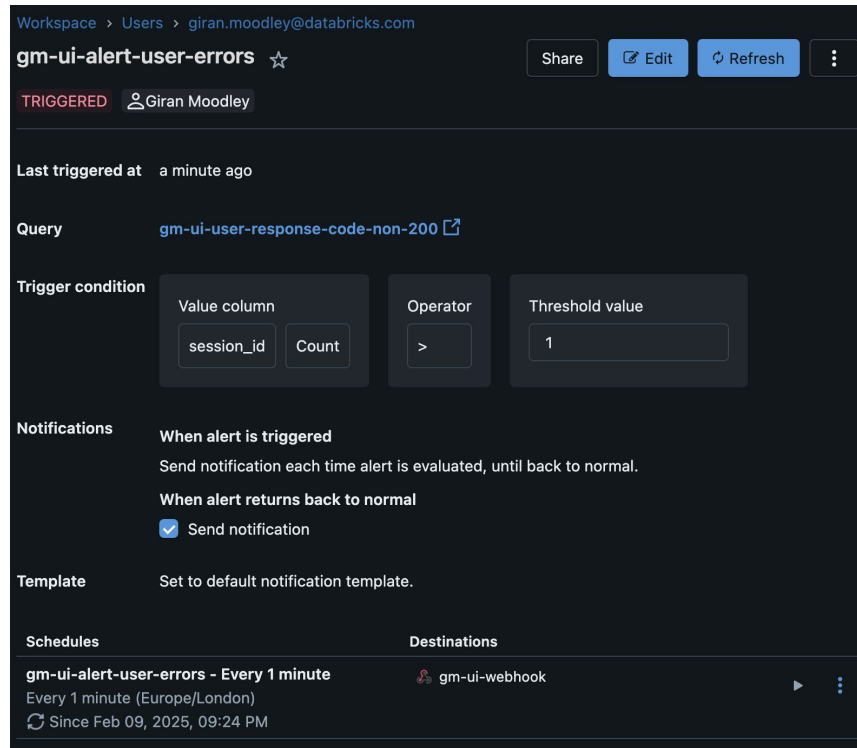
Learns Your Data + Semantics

Controls for Guidance + Oversight



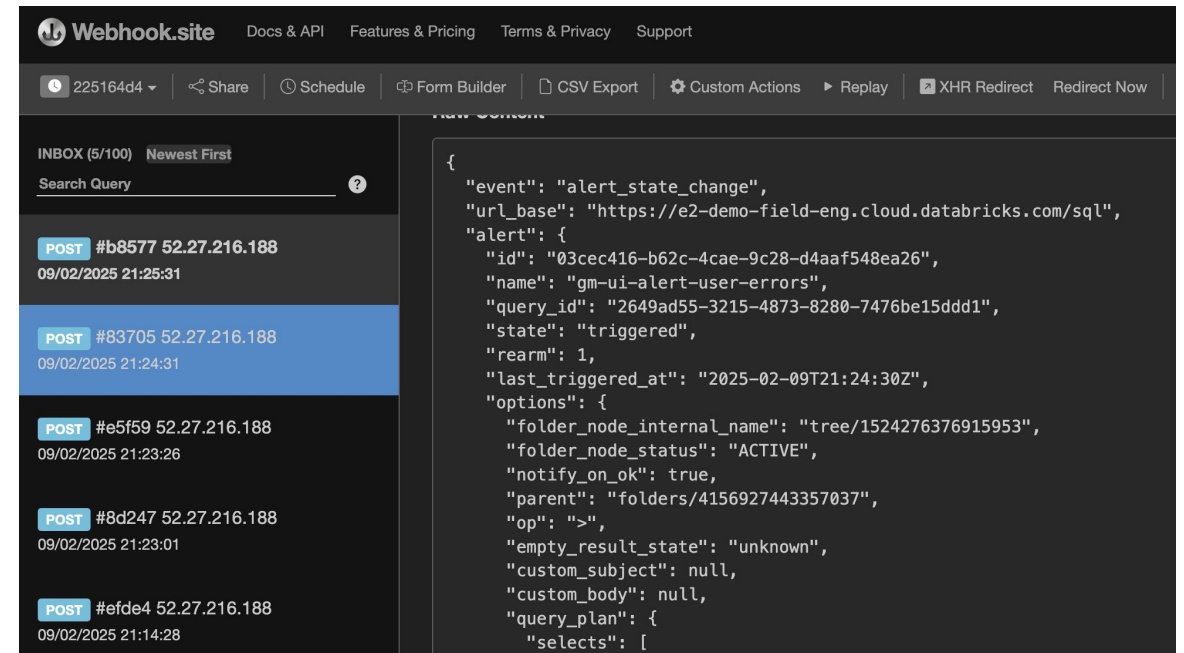
Alerts & Notifications

Respond to significant events in near real-time



The screenshot shows the Databricks Alerts configuration page for an alert named "gm-ui-alert-user-errors". The alert is currently in a "TRIGGERED" state. The configuration includes:

- Query:** gm-ui-user-response-code-non-200
- Trigger condition:** Value column: session_id, Operator: >, Threshold value: 1
- Notifications:** When alert is triggered: Send notification each time alert is evaluated, until back to normal. When alert returns back to normal: ☒ Send notification
- Template:** Set to default notification template.
- Schedules:** gm-ui-alert-user-errors - Every 1 minute (Europe/London) Since Feb 09, 2025, 09:24 PM
- Destinations:** gm-ui-webhook



The screenshot shows the Webhook.site interface. On the left, there is a list of incoming POST requests. The selected request is from 52.27.216.188 at 09/02/2025 21:24:31. On the right, the JSON payload of the selected request is displayed:

```
{
  "event": "alert_state_change",
  "url_base": "https://e2-demo-field-eng.cloud.databricks.com/sql",
  "alert": {
    "id": "03cec416-b62c-4cae-9c28-d4aaf548ea26",
    "name": "gm-ui-alert-user-errors",
    "query_id": "2649ad55-3215-4873-8280-7476be15ddd1",
    "state": "triggered",
    "rearm": 1,
    "last_triggered_at": "2025-02-09T21:24:30Z",
    "options": {
      "folder_node_internal_name": "tree/1524276376915953",
      "folder_node_status": "ACTIVE",
      "notify_on_ok": true,
      "parent": "folders/4156927443357037",
      "op": ">",
      "empty_result_state": "unknown",
      "custom_subject": null,
      "custom_body": null,
      "query_plan": {
        "selects": [
```

Use experience or contextual knowledge from Unity Catalog to curate queries that provide insights. For “real-time” events that require attention, use Databricks Alerts to notify via various destinations (e.g. Slack, Email, PagerDuty, etc)

Demo

Operations of Productionized
Databricks Apps



**I want to grow app
further and
beyond?**

**What will
Databricks Apps
offer in future?**



Summary and Roadmap



Sweet spot for Databricks Apps

Fit

- Exploration apps for your data
- Business logic on top of Data / AI services
- Data-based API services
- Chatbots / RAG / LLM interfaces
- Complex, custom visualizations
- Exposing curated sets of Databricks APIs/actions to specific user groups

Probably won't fit

- Mobile apps
- Static content hosting
- [**Not yet**] Externally facing / public app



Recap

Apps as a product surface

- **Apps are GA!**
- Apps elevate data and AI experience for end users
- Great tool for both Data, ML and Software Engineers

Apps as a business driver

- Easy to build
- Easy to operate
- Easy to scale

Build flexible UIs/APIs and get best value from your data and AI services!



Roadmap Themes



Scalability

- Larger app instance types
- Horizontal scaling
- Auto-stopping



Observability

- Turn-key App observability
- Resource consumption monitoring
- App availability/uptime



Governance

- Git-backed Apps
- App System Tables
- Customized Templates



Getting started with Databricks Apps



Explore samples on
apps-cookbook.dev



Read the [getting started guide and docs](#)



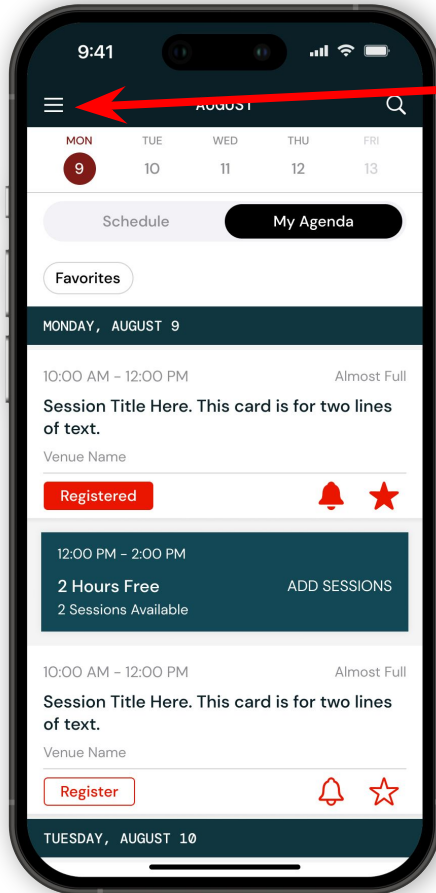
Watch a [5 minute demo](#)



Reach out to your
[Databricks account team](#)

Complete Your Surveys

Your feedback has a direct impact on Data + AI Summit content



- You will receive a survey for each session attended
- Open the Databricks Events app and select “My Surveys” from the menu
- Surveys can also be submitted in the Attendee Portal

databricks

DATA+AI

SUMMIT

